University of HUDDERSFIELD
Inspiring global professionals

UNDERGRADUATE
ASSESSMENT BRIEF

Module Code	CFS2155
Module Title	Computer Programming
Module Leader	Haydn Martin
Module Tutors	Click or tap here to enter text.
Assessment Type (Should match MSD)	Written Assignment
Academic Year	25/26
Term Assessment Applies to	Term 2 - Term 3
Main or Resit	Main

Assessment weighting	60%
Group/Individual	Individual
Word Count/Duration	2400
Module Learning Outcomes	ML01, ML02, ML03

Arrangements For the Return of Work and Feedback	
Submission Date	4/5/2026
Feedback Date	11/05/26
Submission Time (if applicable) If you have any technical issues submitting your work, please contact the Module Leader as soon as possible.	11:59 AM
Submission Method	Brightspace

Assignment Specific Resources	
Software	MS Visual Studio
Equipment	None

Guidance on AI Usage.	
☒	**Level 1- Not Permitted** The use of AI tools is not permitted in any part of this assessment.

☐	**Level 2 – Some use Permitted** Some use of AI tools is permitted in the research/early stages of this assignment, but you must ensure that the work you submit is your own. If you use AI tools, you should acknowledge or reference this in your work. Use the Text reference builder to learn how to reference AI generated ideas. The sorts of questions to consider when using AI are: • Is it accurate? • Are the references genuine? • Has it reproduced bias?
☐	**Level 3 – Integrated** The use of AI tools is integrated in this assessment. Further guidance is included in this assessment brief. Ensure any AI generated material is clearly identified and referenced both within the assignment and in the reference list, using: Text reference builder Image reference builder

General Study Guidance

- Cite all information used in your work which is clearly from a source. Try to ensure that all sources in your reference list are seen as citations in your work, and all names cited in the work appear in your reference list.

- Reference and cite your work in accordance with the APA 7th system – the University's chosen referencing style. For specific advice, you can talk to your subject librarians or go to the library help desk, or you can access library guidance via the following link: APA 7th Referencing Guide

- The University has regulations relating to Academic misconduct, including **plagiarism.** The Academic Skills Team can advise and help you with how to avoid 'poor scholarship' and potential academic misconduct.

- If you have any concerns about your writing, referencing, research or presentation skills, you are welcome to consult the Academic Skills Team, you can book tutorial appointments with them via the website How to book a tutorial appointment

- Further study resources including the Academic Skills Team overview can be found here: Study resources

- Do not exceed the word limit/time/other limit.

Exceptional Circumstances

If you wish to make an EC claim against this module, you can access details on the procedure for claiming ECs, on the Registry website; Consideration of Personal Circumstances - University of Huddersfield

Extensions/Late Submission

If you wish to submit an extension on this module, you can access details on the procedure for submitting extensions on the website: Extensions for Taught Students - University of Huddersfield

Support and Guidance
General help, support and guidance information for your course and time at University can be found here or you can speak to your module leader or Personal Academic Tutor for School based support.
Regulations
The regulations governing assessments can be found here

Assessment Task
CFS2155 Coding Project Assignment The aim of this assignment is to test your ability to design a coded solution in C/C++ to complete a specified task. You will also be required to produce a technical report describing the process of designing, developing, testing your solution.
Task Specific Guidance
A common requirement when working with datasets is the need to sort a values in order of magnitude e.g. highest to lowest. There are a range of algorithms that can be applied to achieve this goal. Once of the simplest approaches is to use a bubble sort algorithm. This is a simple to implement algorithm but suffers from low efficiency compared to other approaches. In this assignment you are required to develop a computer programme in C/C++ that will take do the following: 1. Prompt the user for a filename containing a dataset (this will be supplied); 2. Read sampled data from a supplied file and store in memory; 3. Perform a bubble sort on the dataset; 4. Calculate and display the following values relating to the dataset: length, mean, median, mode, minimum, and maximum;

5. Write the sorted data to a filename of the users choice;
6. Inform the user of the success (or otherwise) of the operation of any file reads/writes.

Arriving at an effective solution will require the following activities:

1. Breakdown of the problem through careful planning and stepwise refinement;
2. Development and debugging of your coded solution;
3. Development of a test plan and subsequent validation of your coded solution.

While the core aim of the assignment is to generate a working coded solution, the primary deliverable is a 2400 word written report that must comprise the following sections which must describe:

- Initial planning and the process of stepwise refinement;
- Code design including data storage, functionalisation, structure, algorithm development;
- Description of the process for testing and validation;
- Consideration of further optimisations e.g. alternative sorting algorithms
- Annotated code listing(s).

Extra credit will be achieved through the implementation of the bubble sort using pointer arithmetic.

<u>Marking Scheme</u>

Written Report (60 marks total)

Criterion	**Mark**
Introductory sections	10
Stepwise refinement	10
Description of code design	10
Testing and debugging	10
Summary sections	10
Communication	10

Coded Solution (40 marks total)

Criterion	Mark
Code structure	20
Code efficiency	10
Code layout and annotation	10

See next section for detailed marking rubrics

Marking Rubrics

Written Report (60 marks)

Criterion / Mark	Excellent (9-10)	Good (6-8)	Fair (4-5)	Poor (0-3)	Marks
Introductory sections	Excellent, summary of the problem and basic theory relating to the coding task which is clear, concise and comprehensive.	Good summary of the problem and basic theory relating to the coding task which is clear, concise with only minor omissions.	Fair summary of the problem and basic theory of the coding task which is clear overall but may contain some omissions.	Poor summary of the problem and theory of coding task which is either very unclear and/or contains many omissions.	**10**
Stepwise refinement	The coding task has been broken down clearly and effectively. Multiple iterations of stepwise refinement have been evidenced with an optimal solution finally being arrived at.	The coding task has been broken down clearly. Multiple iterations of stepwise refinement have been evidenced with a sensible solution finally being arrived at.	The coding task has been broken down, but there may be omissions and/or errors. Stepwise refinement has been evidenced, but this is limited and/or errors in the solution may be present.	The coding task not been broken down. Stepwise refinement has either not been evidenced or there may be copious errors in the proposed solution.	**10**
Description of code design	A detailed breakdown of the code design is present. Code structure described very effectively. Design decisions around datatypes well-justified. The approach to functionalisation of the code is excellently described. Excellent use of flow charts, hierachy diagrams etc. to efficiently convey information.	A detailed breakdown of the code design is present. Code structure described well. Design decisions around datatypes are justified. The approach to functionalisation of the code is described. Good use of flow charts, hierachy diagrams etc. to efficiently convey information.	A breakdown of the code design is present but may be limited or in error in places. Code structure is described to some degree but it also. Design decisions around datatypes are not well-justified. The approach to functionalisation of the code may be present, but is limited or in error. Limited use of flow charts, hierachy diagrams etc. to convey information with minimal word count.	A detailed breakdown of the code design is not present. Code structure is not described vey effectively and is either confusing of error prone. Design decisions around datatypes are not justified. The approach to functionalisation of the code is missing or poorly described. No use of graphical representation e.g. flow charts, hierachy diagrams to convey information or if present they are of very poor quality.	**10**
Testing and debugging	A detailed, comprehensive and logical test plan has been developed which effectively tests all key aspects of the code design. Evidence of the correct operation of the code is fully convincing.	A logical test plan has been developed which effectively tests all key aspects of the code design. Evidence of the correct operation of the code is broadly convincing though there may be an occasional gap/omisson.	A test plan has been developed which tests some aspects of the code design but not all. Some evidence of the correct operation of the code is presented but may still leave concerns.	A logical test plan which tests some aspects of the code design has either not been developed or fails to test key aspects of operation effectively. Evidence of the correct operation of the code is either missing or unconvincing.	**10**
Summary sections	The conclusion summarises the outcomes of the exercise very effectively. Success has been appraised objectively against the original specification. There is an excellent level of self-reflection on what has been learned through the process of completing this assignment. A comprehensive consideration of potential optimisations has been considered.	The conclusion summarises the outcomes of the exercise to a good standard. Success has been appraised against the original specification. Good self-reflection on what has been learned through the process of completing this assignment. Some consideration of potential optimisations has been considered.	The conclusion attempts to summarise the outcomes of the exercise but falls short at times. Appraisal against the original specification may be limited. There is some attempt at self-reflection on what has been learned through the process of completing this assignment. Limited consideration of potential optimisations has been considered.	The conclusion does not summarise the outcomes of the exercise effectively. Success has been not apparised objectively against the original specification, or very poorly. There is an no self-reflection on what has been learned through the process of completing this assignment. No consideration of potential optimisations has been considered.	**10**
Communication	The report achieves a very high standard in all of the following areas: structure, clarity, presentation, effective use of figures, standard of English, academically rigorous referencing.	The report achieves a high standard in the majority of the following areas: structure, clarity, presentation, effective use of figures, standard of English, academically rigorous referencing.	The report achieves a fair standard in the majority of the following areas: structure, clarity, presentation, effective use of figures, standard of English, academically rigorous referencing.	The report fails to attain even the basic standard for a work at this level in any of the following areas: structure, clarity, presentation, effective use of figures, standard of English, academically rigorous referencing.	**10**

(see overleaf for coded solution grading rubric)

Coded Solution (40 marks)

Criterion / Mark	Excellent (17-20)	Good (12-16)	Fair (8-12)	Poor (0-7)	Marks
Code Stucture	Multi-parameter functions and return values used to enhance configurability and maximise code reuse. Sensible variable scope to maximise data safety is used throughout. Advanced use of loop variables and operators unlock patterns and minimise code repetition. Branching code implementation is logical and chosen with clarity in mind. Effective and logical use of multiple library files implemented.	Single-parameter functions are used effectively to enable substantial code reuse. Consideration has been given to sensible variable scope to maximise data safety, with some exceptions. Effective use of loops to minimise code repetition. Branching code implementation is logical throughout. The ability to implement a library files is demonstrated.	Some function usage but it is rather simplistic e.g. no parameters implemented, thus limiting potential for code reuse. Little or no consideration has been given to sensible variable scope to maximise data safety. Some limited use of simple loops to minimise code repetition. Branching code implementation may be confusing at times. A library file has not been included with the project or not implemented correctly.	Function implementation is either very limited or non-existent. No consideration has been given to sensible variable scope to maximise data safety. Looping use is rudimentary and and substantial code repetition is in evidence. No attempt to use a library file has been made.	**20**
	Excellent (9-10)	**Good (6-8)**	**Fair (4-5)**	**Poor (0-3)**	**Marks**
Code Efficiency	Code is written very efficiently with very little room for further improvement. Substantial efforts have been made to reduce the code size without impacting on readability. Virtually no avoidable code repetition present. Data type choice is at all times optimal.	Code generally efficient. Efforts have been made to reduce size of the code though some avoidable repetition is still present.Variable type choice is fairly optimal with some minor exceptions.	Code is either moderately efficient OR generally efficient with some major exceptions. Only minor efforts has been made to reduce size of the code. Unnecessary code repetition is noticeable throughout. Variable type choice is often non-optimal and wasteful.	Code is inefficient overall. There is a large amount of unnecessary code repetition brought about by a lack of consideration in the design. Variable type choice consistently non-optimal.Little or no consideration of good practice is evidenced with respect to developing efficient code.	**10**
Code layout & annotation	*All of the following attributes are implemented to a high standard:* Full header containing filename, author, version number and description. Consistent and neat indentation. Comprehensive yet concise commenting. Function descriptors.	*Most of the following attributes are implemented to a good standard:* Full header containing filename, author, version number and description. Consistent and neat indentation. Comprehensive yet concise commenting. Function descriptors.	*Some of the following attributes are implemented to a fair standard:* Full header containing filename, author, version number and description. Consistent and neat indentation. Comprehensive yet concise commenting. Function descriptors.	*Few/none of the following attributes implemented, generally to a poor standard:* Full header containing filename, author, version number and description. Consistent and neat indentation. Comprehensive yet concise commenting. Function descriptors.	**10**